

GUIA PARA QUEM ESTÁ COMEÇANDO

O projeto do scanner de segurança, explicado do zero

O que ele faz, como funciona por dentro, cada ferramenta usada (inclusive o Docker) e cada conceito importante — sem pressupor que você já programou.

Repositório `~/scan_site` · etapas 4, 5 e 6 do fluxo de scan · outubro de 2026

Como usar este guia

Este documento foi escrito para quem nunca desenvolveu software. Ele vai do mais geral (o que o produto faz) para o mais específico (o que cada pasta do projeto contém). Você não precisa ler tudo de uma vez: os capítulos 1 a 3 dão a visão geral; os capítulos 4 a 6 explicam as ferramentas; os capítulos 7 a 10 mostram como tudo se encaixa no nosso código. No final há um glossário com todos os termos.

Ao longo do texto aparecem três tipos de caixa:

Analogia

Uma comparação com algo do dia a dia, para dar intuição antes do detalhe técnico.

Atenção

Algo que pode dar errado, ou uma regra que não pode ser quebrada.

Em uma frase

O resumo do que importa lembrar.

Sumário

1. O que é o produto
2. As 7 etapas do scan — e quais são nossas
3. Como a web funciona (o mínimo necessário)
4. Conceitos de segurança
5. As ferramentas de segurança: ZAP, nuclei e naabu
6. As ferramentas de desenvolvimento: Python, uv, Git, VS Code, WSL e companhia
7. Docker, explicado com calma
8. Como as peças conversam: filas, banco de dados e o "contrato"
9. Passeio pelas etapas 4, 5 e 6 por dentro
10. A inteligência artificial (Claude) no relatório
11. As regras de ouro e como o código as garante
12. Mapa de pastas e arquivos
13. Como rodar o projeto, passo a passo
14. O que ainda está pendente
15. Glossário

1. O que é o produto

O produto é um **scanner de segurança automatizado vendido como serviço** (o que o mercado chama de *SaaS*, "software como serviço": o cliente usa pela internet e paga uma assinatura, sem instalar nada).

O cliente informa o endereço do site dele (por exemplo, `loja-do-cliente.com.br`). O sistema então, de forma automática:

1. confirma que o site é mesmo daquele cliente;
2. descobre o que está exposto na internet;
3. testa o site procurando falhas de segurança conhecidas;
4. **confirma** quais falhas são reais (e descarta alarmes falsos);
5. entrega um **relatório em linguagem simples**, com o passo a passo para corrigir cada falha;
6. repete tudo periodicamente e mostra a evolução ("3 falhas novas, 5 corrigidas").

Analogia

Pense numa vistoria de imóvel. O vistoriador (o scanner) só entra com autorização do dono, percorre todos os cômodos (reconhecimento), testa portas, janelas e fiação (testes), confirma o que é problema de verdade e o que é só aparência (validação) e entrega um laudo que o dono entende, dizendo o que consertar e como (relatório).

Por que alguém pagaria por isso?

Já existem ferramentas gratuitas que encontram falhas. O problema delas é duplo: (1) apontam muitas falhas que não existem de verdade — os **falsos positivos** — e (2) falam uma língua que só especialistas entendem. O diferencial deste produto está justamente nas duas etapas que este repositório implementa: **validar** (provar que a falha é real) e **explicar e ensinar a corrigir**.

Em uma frase

O produto encontra falhas de segurança em sites, prova quais são reais e ensina o cliente a corrigi-las.

2. As 7 etapas do scan — e quais são nossas

O processo completo tem sete etapas em sequência. A saída de uma é a entrada da seguinte, como uma linha de montagem.



Etapa	O que faz	Quem cuida
1 Autorização	Confirma que o site pertence ao cliente (ex.: pedindo que ele publique um código secreto no DNS do domínio) e define o escopo : o que pode e o que não pode ser testado.	Outro membro do time
2 Reconhecimento	Usa a ferramenta naabu para descobrir quais "portas" do servidor estão abertas e que serviços respondem nelas.	Outro membro do time
3 CVEs	Usa a ferramenta nuclei para comparar o que foi encontrado com milhares de falhas já conhecidas e catalogadas.	Outro membro do time
4 Web	Usa o OWASP ZAP para navegar pelo site e testar cada página e formulário.	Este repositório
5 Validação	Junta os achados repetidos, refaz um teste seguro para provar cada falha e define a gravidade.	Este repositório
6 Relatório	Escreve, para cada falha, "o que é / por que importa / como corrigir", com ajuda de IA, e gera o PDF.	Este repositório
7 Recorrência	Agenda novos scans e compara com os anteriores.	Outro membro do time

Como cada pessoa do time cuida de etapas diferentes, precisamos de um **combinado** sobre o formato dos dados que passam de uma etapa para outra. Isso se chama **contrato** e está explicado no capítulo 8. Graças a ele, cada um pode até usar uma linguagem de programação diferente.

Atenção — a etapa 1 é a fronteira legal

Testar a segurança de um site sem autorização formal do dono é crime. Por isso, nada nas etapas 4, 5 e 6 roda sem a confirmação de que a etapa 1 aprovou o alvo. Isso é verificado no código (capítulo 11).

3. Como a web funciona (o mínimo necessário)

Para entender o que o scanner testa, é preciso entender como um site conversa com o navegador.

HTML, JavaScript e SPA

HTML é a linguagem que descreve a página (títulos, textos, botões). **JavaScript** é a linguagem de programação que roda dentro do navegador e deixa a página interativa. Muitos sites modernos são **SPAs** (*Single Page Applications*): a página é montada pelo JavaScript no navegador. Isso importa porque um robô que só lê o HTML não "enxerga" esses sites; por isso o ZAP tem um segundo rastreador que usa um navegador de verdade (o AJAX spider).

Portas e DNS

Um servidor pode oferecer vários serviços, cada um numa **porta** numerada (como apartamentos num prédio): 80 para HTTP, 443 para HTTPS, 5432 para o banco Postgres etc. O **DNS** é a "lista telefônica" da internet: traduz `loja.com.br` no endereço numérico (IP) do servidor.

4. Conceitos de segurança

Vulnerabilidade

Uma **vulnerabilidade** (ou falha) é um defeito que permite a alguém fazer o que não deveria: ler dados alheios, se passar por outro usuário, alterar informações. Um **achado** (*finding*) é o registro de uma possível vulnerabilidade encontrada pelo scanner.

CVE, CWE e OWASP Top 10 — três "catálogos"

Nome	O que cataloga	Exemplo
CVE	Falhas específicas em produtos específicos ("a versão 2.4.49 do Apache tem tal falha"). Cada uma ganha um número.	CVE-2021-41773
CWE	Tipos de falha, independentemente do produto ("não escapar dados na página"). Usamos o CWE para agrupar e para escolher como validar.	CWE-79 = XSS; CWE-89 = SQL injection
OWASP Top 10	Lista das 10 categorias de risco mais importantes em aplicações web, mantida pela fundação OWASP. A versão vigente é a de 2025. O relatório mapeia cada achado para uma categoria.	A05:2025 = Injeção

Candidato, confirmado, falso positivo

As ferramentas automáticas "desconfiam" de muita coisa. Por isso o projeto usa estados (o campo `status`):

Status	Significado
<code>candidate</code>	Candidato: uma ferramenta apontou, ninguém verificou ainda. É o que sai das etapas 3 e 4.

<code>confirmed</code>	Confirmado: refizemos um teste e provamos que a falha existe.
<code>likely</code>	Provável: há indício forte, mas a prova não é 100% conclusiva.
<code>unconfirmed</code>	Não confirmado: não conseguimos provar com segurança. Continua no relatório , sinalizado.
<code>false_positive</code>	Falso positivo: uma checagem objetiva mostrou que a falha não existe (ex.: o cabeçalho "ausente" está, na verdade, presente).

Atenção — por que "não confirmado" não vira "falso positivo"

Não conseguir provar uma falha não significa que ela não existe. Se descartássemos tudo que não conseguimos provar, esconderíamos falhas reais do cliente. Por isso, só uma checagem determinística (um teste objetivo, que dá sempre o mesmo resultado) ou uma pessoa podem marcar `false_positive`. A IA nunca pode.

Severidade

A gravidade de cada falha: **crítica, alta, média, baixa** ou **informativa**. Ela define a ordem no relatório: o cliente deve corrigir primeiro o que é mais grave.

Autorização e escopo

O **escopo** é a lista do que pode ser tocado: quais hosts (ex.: `loja.com.br`), quais caminhos podem ser incluídos e quais devem ser excluídos (ex.: `/logout`, para não derrubar a sessão; ou uma página que apaga dados). Sair do escopo pode significar atacar um site de terceiros — por isso o projeto checa o escopo em várias camadas.

Scan passivo e scan ativo

- **Passivo:** só observa as respostas que o site já deu, sem enviar nada estranho. Ex.: "esta resposta não tem o cabeçalho de segurança X". É seguro e leve.
- **Ativo:** envia requisições modificadas de propósito para ver como o site reage (ex.: colocar um apóstrofo num campo para ver se o banco de dados dá erro). É o que encontra as falhas mais graves, mas consome recursos e precisa de cuidado para não causar dano.

As falhas que o projeto já sabe validar

XSS refletido (Cross-Site Scripting) — CWE-79

O site pega um valor da URL e o coloca na página sem tratar. Se o valor for código, o navegador o executa. Um atacante manda para a vítima um link com código malicioso escondido no parâmetro; ao abrir, o código roda com a sessão da vítima. **Como provamos:** enviamos um marcador único e inofensivo (algo como `zx3fa1"<zxtag>`) e vemos se ele volta "cru" no HTML. Se volta, está confirmado. A correção é **escapar** a saída: transformar `<` em `<`, para o navegador mostrar o texto em vez de interpretá-lo.

SQL injection — CWE-89

Sites guardam dados em bancos que usam a linguagem **SQL**. Se o site monta a consulta colando o valor digitado pelo usuário, alguém pode digitar pedaços de SQL e mudar a consulta — e ler ou apagar dados. **Como provamos**, sem risco: enviamos o valor normal, depois o mesmo valor com um apóstrofo (`1'`) e depois com dois (`1''`). Se só o apóstrofo único gera uma mensagem de erro do banco, a consulta está sendo montada de forma insegura. Nunca usamos comandos que leem ou alteram dados.

Cabeçalhos de segurança ausentes

Instruções que o servidor dá ao navegador: não deixar a página ser embutida em outro site (`X-Frame-Options` , contra *clickjacking*), não "adivinhar" o tipo de arquivo (`X-Content-Type-Options`), usar sempre HTTPS (`Strict-Transport-Security`), limitar de onde scripts podem vir (`Content-Security-Policy`). **Como provamos**: pedimos a página e olhamos o cabeçalho. É um teste totalmente objetivo — por isso, aqui, podemos marcar falso positivo.

Cookies sem proteção

Cookies podem ter atributos de proteção: `HttpOnly` (scripts não conseguem lê-lo), `Secure` (só trafega por HTTPS) e `SameSite` (não é enviado em requisições forjadas por outros sites). Checamos o cabeçalho `Set-Cookie` diretamente.

5. As ferramentas de segurança: ZAP, nuclei e naabu

As três são **código aberto** (o código é público) com licenças que permitem embuti-las num produto comercial.

OWASP ZAP — a ferramenta da nossa etapa 4

O **ZAP** (*Zed Attack Proxy*) é um programa que testa a segurança de aplicações web. Normalmente uma pessoa o usa com tela, mas ele também roda em **modo daemon** (sem tela, em segundo plano) e é controlado por uma **API** — um conjunto de comandos que outro programa pode enviar pela rede. É assim que o nosso código o dirige.

Analogia — o que é uma API

Uma API é como o cardápio e o garçom de um restaurante: você não entra na cozinha; escolhe um item do cardápio (um comando, como "rastreie este site") e o garçom traz o resultado. O ZAP é a cozinha; nosso código faz os pedidos.

O que o nosso código pede ao ZAP, em ordem:

1. **Sessão nova**: limpa tudo do scan anterior, para os alertas de dois clientes nunca se misturarem.
2. **Modo protegido e contexto**: dizemos ao ZAP quais endereços estão no escopo; em modo protegido ele se recusa a atacar qualquer coisa fora dele.
3. **Limites**: quantas conexões simultâneas, intervalo entre requisições e tempo máximo.

4. **Política**: quais testes ativos estão ligados. Começamos com todos desligados e ligamos apenas uma lista de testes considerados seguros (ver capítulo 11).
5. **Spider** (rastreador): segue os links do site para descobrir as páginas. O **AJAX spider** faz o mesmo usando um navegador real, para sites feitos em JavaScript.
6. **Scan passivo**, depois **scan ativo**.
7. **Coleta dos alertas**, com a requisição e a resposta que expuseram cada um.

nuclei — etapa 3 (de outro colega)

Uma ferramenta que testa milhares de "receitas" (*templates*) de falhas conhecidas. Os achados dela chegam até nós como candidatos, no mesmo formato dos do ZAP, e passam pela mesma validação.

naabu — etapa 2 (de outro colega)

Um **scanner de portas**: descobre quais portas estão abertas num servidor. Não usamos diretamente, mas as rotas descobertas nas etapas 2 e 3 podem ser passadas ao ZAP como ponto de partida.

Os alvos de laboratório: Juice Shop e DVWA

Para testar o scanner sem tocar em sites reais, usamos sites **propositalmente vulneráveis**, criados para treino: o **OWASP Juice Shop** (uma loja falsa cheia de falhas) e o **DVWA** (*Damn Vulnerable Web Application*). Eles rodam no seu próprio computador, isolados da internet.

Atenção

Testes, exemplos e dados de exemplo usam **somente** esses alvos de laboratório. Nunca um site real, nem "só para ver".

6. As ferramentas de desenvolvimento

Python — a linguagem de programação

Todo o nosso código está escrito em **Python** (versão 3.13), uma linguagem conhecida por ser legível. Um programa em Python é um conjunto de arquivos de texto terminados em `.py`. Um exemplo real do projeto, simplificado:

```
# Se o cabeçalho de segurança está presente, é falso positivo;
# se está ausente, a falha é confirmada.
if cabeçalho_ok(resposta):
    return resultado(FALSE_POSITIVE, "cabeçalho presente e válido")
return resultado(CONFIRMED, "cabeçalho ausente ou inválido")
```

Alguns termos de Python que aparecem no projeto:

- **Função**: um bloco de código com nome, que recebe dados e devolve um resultado. Cada validador é uma função.

- **Módulo:** um arquivo `.py`. **Pacote:** uma pasta de módulos. Nosso pacote se chama `scanner`.
- **Biblioteca / dependência:** código escrito por outras pessoas que reaproveitamos (ex.: a biblioteca que fala com o ZAP).
- **Tipos:** anotações dizendo que tipo de dado cada coisa é (texto, número, lista...). Ajudam a pegar erros antes de rodar.

As bibliotecas que usamos

Biblioteca	Para que serve no projeto
zaproxy	Conversar com a API do ZAP.
httpx	Fazer requisições HTTP (as provas da validação).
Pydantic	Definir o formato dos dados (um "molde") e rejeitar automaticamente o que não segue o molde. Ex.: um achado sem URL é recusado.
redis	Falar com o Redis, a fila de mensagens entre as etapas.
SQLAlchemy e psycopg	Falar com o banco de dados Postgres.
Alembic	Criar e evoluir as tabelas do banco com histórico (as "migrações").
anthropic	SDK oficial para chamar a IA Claude (capítulo 10).
Jinja2	Preencher um modelo de página HTML com os dados do relatório (como uma mala direta).
WeasyPrint	Transformar o HTML em PDF. (Este guia foi gerado com ela.)

uv — o gerenciador do projeto

O **uv** instala a versão certa do Python e de todas as bibliotecas, numa pasta isolada do projeto chamada `.venv` (ambiente virtual), sem bagunçar o resto do computador. A lista do que precisamos está no arquivo `pyproject.toml`; as versões exatas ficam travadas no `uv.lock`, para todo mundo ter exatamente o mesmo ambiente.

```
uv sync      # instala tudo o que o projeto precisa
uv run pytest # roda um comando dentro do ambiente do projeto
```

Ferramentas de qualidade: pytest, ruff e mypy

- **pytest** roda os **testes automatizados**: pequenos programas que verificam se o código faz o que deveria. Ex.: "se a página devolve o marcador sem escape, o validador de XSS tem que dizer *confirmado*". Hoje são 45 testes, que rodam em cerca de 2 segundos. Se alguém quebrar algo sem querer, um teste falha e avisa.
- **ruff** é o "corretor ortográfico" do código: aponta erros comuns, padroniza a formatação e acusa práticas inseguras.

- **mypy** verifica os tipos: avisa, por exemplo, se em algum lugar passamos texto onde se esperava número.

Existem dois tipos de teste no projeto: **unitários** (`tests/unit/`), que testam cada peça isolada, com respostas simuladas, sem rede; e de **integração** (`tests/integration/`), que testam contra o Juice Shop de verdade, rodando no laboratório.

Git — o histórico do projeto

O **Git** guarda o histórico de todas as mudanças do código. Cada "foto" salva é um **commit**, com descrição e autor. Dá para voltar atrás, comparar versões e trabalhar em paralelo com o time. O arquivo `.gitignore` lista o que **não** deve entrar no histórico — principalmente o `.env`, que contém senhas.

VS Code, WSL e o terminal

- **VS Code**: o editor onde você abre e edita os arquivos do projeto.
- **Terminal**: a janela onde se digitam comandos de texto (como os que aparecem neste guia).
- **WSL2** (*Windows Subsystem for Linux*): um Linux (Ubuntu) rodando dentro do seu Windows. Usamos Linux porque servidores e o Docker funcionam melhor nele. O projeto mora em `~/scan_site`, **dentro** do Linux — e não no OneDrive nem em `/mnt/c`, onde ficaria lento e com conflitos de sincronização.

Arquivos de configuração e segredos (.env)

Senhas e chaves (do banco, do ZAP, da API da Claude) nunca são escritas no código. Ficam num arquivo `.env`, que existe só no seu computador/servidor e nunca vai para o Git. O arquivo `.env.example` é um modelo que lista quais variáveis existem, sem os valores reais.

7. Docker, explicado com calma

O problema que o Docker resolve

Para o projeto rodar, é preciso Python 3.13, dezenas de bibliotecas, as bibliotecas de sistema do WeasyPrint, o ZAP (que precisa de Java), o Redis e o Postgres — cada um na versão certa, configurado do jeito certo. Instalar tudo isso na mão em cada computador e em cada servidor é demorado e dá o clássico "na minha máquina funciona". O **Docker** resolve isso empacotando cada programa junto com tudo de que ele precisa.

Analogia — o contêiner de navio

Antes dos contêineres, cada carga era embarcada de um jeito (sacos, caixas, barris). O contêiner padronizou: não importa o que vai dentro, o navio, o guindaste e o caminhão sabem lidar com ele. O Docker faz o mesmo com software: não importa se dentro está o ZAP, o Postgres ou nosso código — tudo roda do mesmo jeito em qualquer máquina que tenha Docker.

Imagem e container

- Uma **imagem** é o pacote pronto e congelado: sistema mínimo + programa + dependências. É como uma **receita com os ingredientes já separados**, ou o molde de um bolo.
- Um **container** é uma imagem **em execução**. É o bolo assado a partir do molde. De uma mesma imagem podem sair vários containers.

Cada container é isolado: tem seus próprios arquivos e sua própria rede, e só enxerga o que permitirmos. Isso também é uma camada de **segurança**.

Dockerfile — a receita da nossa imagem

O arquivo `Dockerfile` descreve, passo a passo, como montar a imagem do nosso código:

```
FROM python:3.13-slim          # parte de um Linux mínimo com Python
RUN apt-get install ... libpango ...  # instala o que o WeasyPrint (PDF) precisa
RUN uv sync --no-dev           # instala as bibliotecas do projeto
COPY scanner ./scanner         # copia o nosso código para dentro
RUN useradd ... scanner        # cria um usuário sem poderes de administrador
USER scanner                   # e passa a rodar como ele
```

Um detalhe importante: geramos **uma única imagem** com o pacote `scanner`, e os três "trabalhadores" (etapas 4, 5 e 6) usam essa mesma imagem, cada um com um comando diferente. Menos coisa para manter.

Docker Compose — vários containers juntos

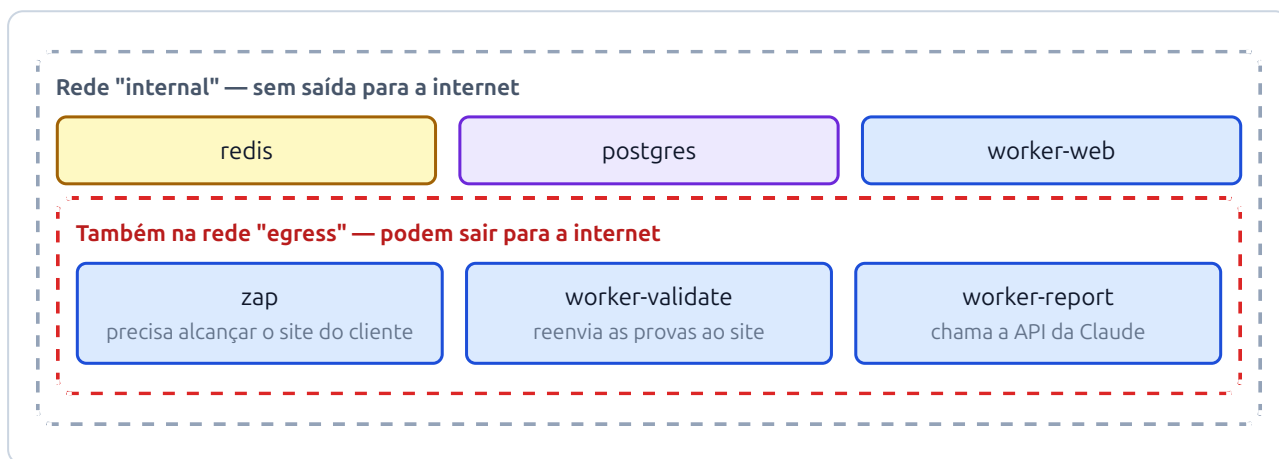
Nosso sistema não é um container só, são vários que precisam conversar. O **Docker Compose** lê o arquivo `docker-compose.yml`, que descreve todos eles, e sobe tudo com um comando: `docker compose up -d`. Os containers (chamados de **serviços**) são:

Serviço	O que é
redis	A fila de mensagens entre as etapas (compartilhada com o time).
postgres	O banco de dados com scans, achados e relatórios (compartilhado com o time).
migrate	Roda uma vez ao subir: cria/atualiza as tabelas do banco e termina.
zap	O OWASP ZAP em modo daemon.
worker-web	Etapas 4: dirige o ZAP.
worker-validate	Etapas 5: deduplica e valida.
worker-report	Etapas 6: escreve e gera o relatório.

Um **worker** ("trabalhador") é um programa que fica rodando o tempo todo, esperando tarefas na fila; quando chega uma, ele a executa e volta a esperar.

Redes: quem pode falar com quem

O Docker permite criar redes virtuais. Usamos duas, por segurança:



Assim, se algum componente for comprometido, ele tem o mínimo de acesso possível. A API do ZAP, em especial, nunca é exposta para fora do computador (em produção): só os workers a enxergam. No laboratório existe ainda uma terceira rede, `lab`, onde ficam o Juice Shop e o DVWA, sem acesso à internet.

Volumes: o que sobrevive

Quando um container é apagado, tudo que estava dentro dele some. Para dados que precisam durar (o conteúdo do banco, por exemplo), usamos **volumes**: pastas guardadas fora do container. No nosso `compose`: `pg-data` (Postgres) e `redis-data` (Redis).

Limites de recursos

O scan ativo do ZAP consome muita memória (cerca de 3 GB). Por isso o container do ZAP tem limite de memória (`mem_limit: 4g`) e de processador (`cpus: 2`), e a máquina virtual Java dele recebe `-Xmx3g` . Sem isso, um scan pesado poderia travar o servidor inteiro.

Por que o ZAP e o worker-web andam em par

Cada `worker-web` tem o seu próprio ZAP e faz **um scan por vez**. Se dois clientes usassem o mesmo ZAP ao mesmo tempo, os alertas de um apareceriam no relatório do outro — um vazamento de dados. Para atender mais clientes ao mesmo tempo, sobe-se mais pares ZAP + `worker-web`.

Docker Desktop e o WSL

No Windows, o Docker vem pelo **Docker Desktop**. Para que ele funcione dentro do Ubuntu do WSL, é preciso ativar a integração: *Docker Desktop* → *Settings* → *Resources* → *WSL integration* → *ligar o Ubuntu*.

Atenção — situação atual

No momento em que este guia foi escrito, o comando `docker` não estava disponível no seu Ubuntu, ou seja, essa integração ainda não está ligada. Até lá, dá para rodar os testes unitários com o `uv`, mas não subir os containers.

Em uma frase

O Docker empacota cada peça do sistema com tudo de que ela precisa; o Compose sobe todas juntas, cada uma na rede certa e com os limites certos.

8. Como as peças conversam: filas, banco e o "contrato"

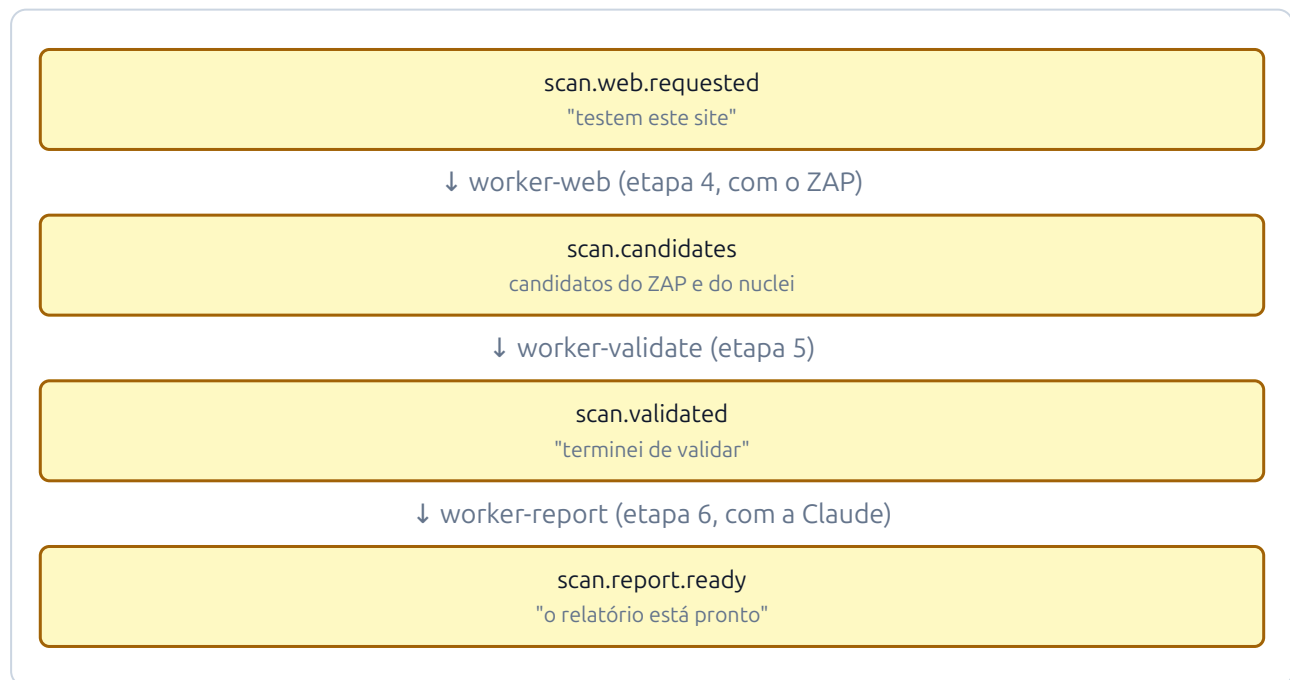
A fila (Redis Streams)

As etapas não chamam umas às outras diretamente. Cada uma deixa uma **mensagem** numa fila, e a etapa seguinte a pega quando puder. Usamos o **Redis**, um programa muito rápido que guarda dados na memória, com o recurso **Streams** (filas de mensagens).

Analogia — a esteira de pedidos da cozinha

O garçom espeta o pedido no trilho; o cozinheiro pega quando está livre. Se o cozinheiro sair por um minuto, os pedidos não se perdem: ficam no trilho. E se ele derrubar um prato no meio do preparo, o pedido continua lá para ser refeito.

As filas oficiais, em ordem:



Vantagens: cada etapa pode cair e voltar sem perder trabalho; dá para colocar mais workers numa etapa lenta; e cada colega do time pode escrever sua etapa em qualquer linguagem.

O formato JSON e o contrato

As mensagens são escritas em **JSON**, um formato de texto simples para dados estruturados, que qualquer linguagem entende. Um achado, resumido, é assim:

```
{
  "finding_id": "9f2c...",           identificador único
  "scan_id": "scan-123",
  "source": { "tool": "zap", "rule_id": "40012" },
  "title": "Cross Site Scripting (Reflected)",
  "cwe": 79, "owasp": "A05:2025",
  "severity": "medium",
  "status": "confirmed",
  "location": { "url": "http://juice-shop:3000/busca?q=x", "method": "GET", "parameter":
"q" },
  "evidence": [ { "request": {...}, "response": {...} } ]
}
```

O **contrato** é o documento que define exatamente que campos existem e que valores são aceitos. Ele fica em `contracts/finding.schema.json` (o achado) e `contracts/message.schema.json` (o "envelope" de toda mensagem, que sempre leva `scan_id`, `target_id`, a etapa, a versão do contrato e o escopo). Esse formato de descrição se chama **JSON Schema**.

Atenção

Mudar o contrato quebra o trabalho dos colegas. Toda mudança precisa ser combinada com o time e vem com o número de versão (`schema_version`) incrementado. Há um teste automatizado que garante que o nosso código continua seguindo o contrato.

Idempotência — a mesma mensagem duas vezes não duplica nada

Em sistemas com filas, às vezes a mesma mensagem é entregue duas vezes (por exemplo, se um worker cair logo depois de terminar). **Idempotente** é uma operação que, repetida, dá o mesmo resultado que feita uma vez. Garantimos isso de duas formas:

- Cada mensagem tem um `message_id`; antes de processar, registramos no banco "esta mensagem já foi tratada". Se chegar de novo, é ignorada.
- O `finding_id` de cada achado é calculado a partir dos próprios dados (scan, ferramenta, regra, URL, parâmetro) usando um **hash** — uma "impressão digital" matemática. O mesmo achado sempre gera o mesmo identificador, então não há como ele virar dois.

Mensagens inválidas ou de scans não autorizados vão para uma fila separada, de rejeitadas (*dead-letter*), com os segredos mascarados, para alguém investigar.

O banco de dados (Postgres)

O **PostgreSQL** guarda os dados que precisam durar, organizados em **tabelas** (como planilhas com colunas fixas). Você, Ahmed, é o dono do **schema** (a estrutura das tabelas). As tabelas criadas são:

Tabela	O que guarda
<code>scans</code>	Cada scan, se o alvo foi verificado e o escopo travado. Escrita pela etapa 1; nós só lemos.
<code>findings</code>	Os achados, com status e severidade.

<code>processed_messages</code>	Quais mensagens já foram processadas (idempotência).
<code>stage_progress</code>	Quais ferramentas (ZAP, nuclei) já entregaram seus candidatos para cada scan.
<code>reports</code>	O relatório final em JSON, HTML e PDF.
<code>remediation_cache</code>	Textos de correção já gerados pela IA, para reaproveitar (economia).

Migrações (pasta `migrations/`, ferramenta Alembic) são os "roteiros" de criação e alteração dessas tabelas, numerados em ordem. Assim, qualquer banco, em qualquer máquina, chega exatamente à mesma estrutura. O container `migrate` aplica esses roteiros sempre que o sistema sobe.

9. Passeio pelas etapas 4, 5 e 6 por dentro

Vamos acompanhar um scan do começo ao fim.

Etapa 4 — Teste web (`scanner/web_scan/`)

1. O worker-web pega uma mensagem da fila `scan.web.requested`.
2. **Checa a autorização** no banco (capítulo 11). Se falhar, recusa e registra.
3. Prepara o ZAP: sessão nova, contexto com o escopo, modo protegido, limites e política segura.
4. Se o cliente forneceu uma credencial de teste, o ZAP passa a enviá-la em toda requisição, para testar também a área logada.
5. Adiciona as rotas vindas das etapas 2/3 (só as que estão no escopo), roda spider, AJAX spider, scan passivo e ativo.
6. Se o tempo máximo estourar, interrompe e entrega o que já encontrou.
7. Converte cada alerta do ZAP num achado com `status: candidate`, com a requisição e a resposta (mascaradas), e publica em `scan.candidates`.

Arquivos: `zap.py` (conversa com o ZAP), `policy.py` (lista de testes seguros), `alerts.py` (alerta → achado), `service.py` (orquestra), `__main__.py` (liga o worker).

Etapa 5 — Validação (`scanner/validation/`)

1. O worker-validate pega os candidatos (do ZAP e, separadamente, do nuclei) e checa a autorização.
2. **Deduplicação** (`dedup.py`): se ZAP e nuclei acharam a mesma coisa (mesma URL, método, parâmetro e CWE), vira um achado só, que registra as duas fontes e fica com a maior severidade.
3. **Escolha do validador**: há um validador por família de falha (pasta `validators/`), escolhido pela regra do ZAP, pelo template do nuclei ou pelo CWE.
4. **Prova**: o validador refaz um teste seguro (capítulo 4) usando um cliente HTTP especial (`http.py`) que só aceita GET/HEAD, só dentro do escopo, sem seguir redirecionamentos e com limite de velocidade.
5. Se não existe validador para aquela falha, ou se a prova dá erro, o achado fica `unconfirmed` — nunca é descartado.

6. **Severidade** (`severity.py`): falhas confirmadas recebem a severidade padrão do seu tipo (ex.: SQL injection confirmada = alta). Falhas não confirmadas nunca são rebaixadas.
7. Grava tudo no banco e avisa em `scan.validated` quais ferramentas já terminaram.

Etapa 6 — Relatório (`scanner/report/`)

1. O worker-report só começa quando ZAP e nuclei já entregaram seus candidatos (e só se ainda não houver relatório para aquele scan).
2. Detecta a **stack** do site (as tecnologias: nginx, PHP, Express...) olhando os cabeçalhos das respostas (`stack.py`), para que a correção seja específica.
3. Para cada achado, busca o texto de correção: primeiro no **cache**; se não tiver, pede à **Claude**; se a IA recusar ou falhar, usa o **catálogo estático** (`catalog.py`), com textos prontos em português. Nenhum achado fica sem texto.
4. Monta o relatório em **JSON** (`builder.py`) — a fonte da verdade —, ordenado: confirmados primeiro, depois prováveis e não confirmados, do mais grave ao menos grave. Os falsos positivos ficam numa seção separada, "descartadas por verificação", com o motivo.
5. Gera o **HTML** a partir de um modelo (`templates/report.html.j2`) e o **PDF** a partir do HTML (`render.py`).
6. Grava no banco e publica em `scan.report.ready`.

Atenção — o relatório contém "ataques"

As evidências podem conter código malicioso (justamente o que provou o XSS). Se o HTML do relatório mostrasse isso cru, o próprio relatório seria vulnerável. Por isso o modelo usa **escape automático** de tudo, e o gerador de PDF é proibido de baixar qualquer coisa da internet. Há um teste que verifica isso.

10. A inteligência artificial (Claude) no relatório

O que a IA faz — e o que ela não faz

A IA **só escreve a explicação e a correção** de cada tipo de falha. Ela **não decide** se uma falha existe, não muda status e não descarta nada. Isso é proposital: estudos mostram que modelos de linguagem tendem a "abafar" falhas verdadeiras, e esconder uma falha real do cliente é o pior erro possível para este produto.

Como chamamos a Claude

- **Modelo:** `claude-opus-5-5`. Trocar de modelo é decisão do time.
- **SDK oficial** `anthropic`: a biblioteca da própria Anthropic para Python.
- **Saída estruturada:** em vez de pedir um texto livre e tentar "garimpar" as partes, entregamos à IA um molde (Pydantic) com os campos *o que é*, *por que importa*, *passos para corrigir* (cada um com comando ou código), *como verificar* e *referências*. A resposta já vem nesse formato, validada.

- **Thinking adaptativo e effort:** o modelo decide quanto "pensar" antes de responder; o *effort* (esforço) regula quão profundo ele vai. Usamos `high`, definido explicitamente.
- **Prompts versionados:** as instruções que damos à IA (o **prompt**) ficam em arquivos próprios em `scanner/report/prompts/`, com versão (`remediation_v1`). Mudou o texto, cria-se uma versão nova.

Privacidade: o que vai para a IA

Mandamos **apenas** o tipo da falha (título, CWE, categoria OWASP, regra da ferramenta), a stack detectada e o idioma. **Nunca** a URL do cliente, as evidências, cookies, tokens ou dados pessoais. Ainda assim, todo texto passa pelo mascaramento antes de sair.

Economia: cache

Como o texto de correção depende só do tipo da falha (e não do cliente), um XSS em site nginx em português tem sempre a mesma explicação. Guardamos cada resposta na tabela `remediation_cache`, com a chave (tipo de falha, stack, idioma, versão do prompt). O segundo cliente com a mesma falha não gera custo de IA. Além disso, usamos o **prompt caching** da API, que barateia a parte fixa das instruções quando ela se repete.

Quando a IA recusa

A Claude tem filtros de segurança, e um texto sobre vulnerabilidades pode ser recusado por engano (a resposta volta com `stop_reason == "refusal"`). Nesse caso — e também em erro de rede, limite de uso ou resposta incompleta — o sistema usa o catálogo estático. Recusas não entram no cache, para tentar a IA de novo na próxima vez.

11. As regras de ouro e como o código as garante

O CLAUDE.md lista cinco regras que nunca podem ser quebradas. Veja onde cada uma é protegida:

1. Nenhum scan sem autorização

Arquivo `scanner/common/authz.py`. Antes de qualquer etapa trabalhar, o código confere **no banco** (e não só na mensagem) que: o scan existe; o alvo bate; a etapa 1 marcou o alvo como verificado; o escopo está travado; e o escopo da mensagem é idêntico ao travado no banco (para ninguém "esticar" o escopo no caminho). Se algo falhar, a tarefa é recusada e registrada. Não existe "modo de teste" que pule isso.

2. Nunca sair do escopo

Três camadas: (a) o ZAP recebe um contexto com apenas os endereços permitidos e roda em modo protegido; (b) alertas fora do escopo são jogados fora; (c) o cliente HTTP da validação (`ScopeGuard`, em `scanner/common/scope.py`) recusa qualquer URL fora do escopo e não segue redirecionamentos, que poderiam levar a outro site.

3. Não causar dano

- A política do ZAP liga só uma lista de testes seguros. Ficaram de fora, por exemplo: testes que **gravam dados** no site (XSS persistente), que fazem o site **acessar a internet** (SSRF, Log4Shell), que são **pesados como um ataque de negação de serviço** (estouro de buffer, SQL injection por tempo) e que usam métodos que alteram dados.
- Limites de conexões, de requisições por segundo e de tempo total.
- As provas da validação só leem (GET/HEAD) e usam testes inofensivos.

4. Teste local só contra laboratório

O `docker-compose.lab.yml` sobe o Juice Shop e o DVWA numa rede isolada. Todos os exemplos e testes do projeto usam esses alvos.

5. Dados do cliente são sensíveis

Arquivo `scanner/common/masking.py`. Mascara cabeçalhos sensíveis (Cookie, Authorization...), tokens, senhas, JWTs, e-mails, CPFs e números de cartão. É aplicado nos logs, nas evidências, nas mensagens rejeitadas, no que vai para a IA e no relatório.

Logs

Logs são o "diário de bordo" do programa: registros do que aconteceu, usados para investigar problemas. Os nossos são em JSON (fáceis de pesquisar), sempre trazem o `scan_id` e passam pelo mascaramento antes de serem escritos.

12. Mapa de pastas e arquivos

Caminho	O que é
<code>CLAUDE.md</code>	As instruções do projeto: regras, arquitetura, convenções.
<code>docs/</code>	Documentos (o PDF do processo e este guia).
<code>pyproject.toml</code> , <code>uv.lock</code>	Lista e versões travadas das bibliotecas; configurações do ruff, mypy e pytest.
<code>Dockerfile</code> , <code>.dockerignore</code>	Receita da imagem Docker e o que não entra nela.
<code>docker-compose.yml</code>	Todos os serviços do sistema.
<code>docker-compose.lab.yml</code>	Alvos de laboratório + container de testes de integração.
<code>.env.example</code>	Modelo das configurações e segredos (copiar para <code>.env</code>).
<code>contracts/</code>	O contrato com o time (JSON Schema do achado e da mensagem).
<code>alembic.ini</code> , <code>migrations/</code>	Estrutura do banco e seu histórico.

<code>scanner/common/</code>	Peças compartilhadas: modelos de dados, configuração, fila, banco, autorização, escopo, mascaramento, logs, mapa OWASP.
<code>scanner/web_scan/</code>	Etapa 4 (ZAP).
<code>scanner/validation/</code>	Etapa 5 (dedup, cliente HTTP seguro, severidade, <code>validators/</code>).
<code>scanner/report/</code>	Etapa 6 (catálogo, IA, cache, montagem, <code>prompts/</code> , <code>templates/</code>).
<code>tests/unit/</code>	Testes de cada peça isolada.
<code>tests/integration/</code>	Testes contra o laboratório.
<code>.gitignore</code>	O que nunca entra no Git (<code>.env</code> , ambiente virtual, relatórios locais).

Em cada pasta de etapa, o arquivo `__main__.py` é o "botão de ligar" do worker (é ele que o comando `python -m scanner.validation` executa) e o `service.py` contém a sequência de passos da etapa.

13. Como rodar o projeto, passo a passo

Todos os comandos abaixo são digitados no terminal do Ubuntu (WSL).

Abrir o projeto

```
cd ~/scan_site      # entra na pasta do projeto
code .              # abre no VS Code
```

Rodar os testes e as verificações (já funciona hoje)

```
uv sync              # instala/atualiza as dependências
uv run pytest         # roda os testes unitários
uv run ruff format . && uv run ruff check .  # formata e revisa o código
uv run mypy           # verifica os tipos
```

O resultado esperado do `pytest` é algo como `45 passed, 1 skipped` ("45 passaram, 1 pulado" — o pulado é o de integração, que só roda no laboratório). Se aparecer `failed`, algo quebrou e a mensagem diz onde.

Se o terminal disser que `uv` não existe, rode `source ~/.local/bin/env` (ou abra um terminal novo): o `uv` foi instalado em `~/.local/bin`.

Subir o sistema com Docker (depois de ligar a integração do WSL)

```
cp .env.example .env      # uma vez só; depois edite o .env e troque as senhas
docker compose up -d      # sobe tudo em segundo plano (-d)
docker compose ps          # mostra quais containers estão rodando
docker compose logs -f worker-validate  # acompanha os logs de um serviço
docker compose down        # desliga tudo (os volumes com os dados ficam)
```

Laboratório e testes de integração

```
docker compose -f docker-compose.yml -f docker-compose.lab.yml up -d
docker compose -f docker-compose.yml -f docker-compose.lab.yml --profile test run --rm tests
```

O primeiro comando sobe o sistema mais os alvos vulneráveis; o segundo roda os testes de integração dentro da rede do laboratório (ex.: confirmar a SQL injection conhecida da busca do Juice Shop).

Banco de dados

```
uv run alembic revision -m "descricao" # cria um novo roteiro de migração
```

As migrações são aplicadas automaticamente pelo container `migrate` sempre que o sistema sobe.

14. O que ainda está pendente

Para combinar com o time

1. **Formato do escopo** que vem da etapa 1. Criamos um formato provisório (hosts permitidos, URLs de partida, caminhos incluídos e excluídos); precisa ser confirmado.
2. **Como a etapa 3 (nuclei) avisa que terminou.** Hoje o relatório espera que ZAP e nuclei tenham entregado seus candidatos (configurável em `REPORT_REQUIRED_TOOLS`).
3. **Credenciais de teste.** Hoje vão no conteúdo da mensagem e só no formato de cabeçalho. O ideal é a mensagem levar só uma referência, e a senha ficar num cofre de segredos.

Ainda não testado na prática

- Os containers e a conversa real com o ZAP ainda não rodaram, porque o Docker não está ligado no WSL. Os testes unitários passam, mas o primeiro scan de verdade contra o Juice Shop é o próximo passo importante.

Decisões que podem ser revistas

- **Provas só em GET.** Falhas em formulários POST ficam como "não confirmadas", por segurança. Pode ser ampliado com cuidado.
- **Poucos validadores.** Há validadores para XSS refletido, SQL injection por erro, cabeçalhos e cookies. Outras famílias (path traversal, open redirect...) ainda saem como "não confirmadas" até ganharem o seu.
- **Sem troca automática de modelo de IA quando ele recusa.** A API oferece isso, mas o CLAUDE.md pede o catálogo estático e diz que trocar de modelo é decisão do time.
- **O cache das instruções fixas da IA pode não estar funcionando ainda,** porque o texto é curto (há um tamanho mínimo para o cache funcionar). Dá para conferir pelos logs.

15. Glossário

Achado (finding)

Registro de uma possível falha de segurança.

AJAX spider

Rastreador do ZAP que usa um navegador real, para sites montados em JavaScript.

API

Conjunto de comandos que um programa oferece para outro programa usar.

Autorização

Permissão formal do dono do site para testá-lo (etapa 1).

Banco de dados

Programa que guarda dados de forma organizada e permanente (usamos o Postgres).

Biblioteca / dependência

Código de terceiros reaproveitado no projeto.

Cabeçalho (header)

Linha de informação extra numa requisição ou resposta HTTP.

Cache

Guardar um resultado para reaproveitar em vez de recalcular.

Candidato

Achado ainda não verificado.

Catálogo estático

Textos de correção prontos, usados quando a IA não responde.

Clickjacking

Embutir um site invisível em outro para induzir cliques.

Commit

Uma "foto" salva das mudanças no Git.

Container

Uma imagem Docker em execução, isolada.

Contexto (ZAP)

Definição, dentro do ZAP, de quais endereços pertencem ao alvo.

Contrato

O formato combinado dos dados trocados entre as etapas.

Cookie

Pequeno texto que o site guarda no navegador para lembrar do usuário.

CVE

Identificador de uma falha específica de um produto específico.

CWE

Identificador de um tipo de falha.

Daemon

Programa que roda em segundo plano, sem tela.

Dead-letter

Fila para onde vão as mensagens rejeitadas.

Deduplicação

Juntar achados repetidos em um só.

Determinístico

Que dá sempre o mesmo resultado para a mesma entrada.

DNS

Sistema que traduz nomes de sites em endereços numéricos.

Docker / Docker Compose

Ferramenta que empacota e roda programas em containers / que sobe vários containers juntos.

Dockerfile

Receita para montar uma imagem Docker.

Escapar (escape)

Transformar caracteres especiais para que sejam exibidos como texto, e não interpretados.

Escopo

O que pode e o que não pode ser testado.

Evidência

A requisição e a resposta que mostram a falha.

Falso positivo

Alerta de falha que, verificado, não existe.

Git

Sistema de histórico de versões do código.

Hash

"Impressão digital" matemática de um dado; o mesmo dado sempre gera o mesmo hash.

HTML

Linguagem que descreve o conteúdo das páginas web.

HTTP / HTTPS

Protocolo de conversa entre navegador e servidor / a versão criptografada.

Idempotente

Que pode ser repetido sem mudar o resultado.

Imagem (Docker)

Pacote congelado com um programa e tudo de que ele precisa.

JSON / JSON Schema

Formato de texto para dados estruturados / a forma de descrever as regras desse formato.

LLM

"Modelo de linguagem grande": o tipo de IA da Claude.

Log

Registro do que o programa fez.

Mascarar

Trocar dados sensíveis por `***` ou marcadores.

Método HTTP

Tipo de requisição: GET (ler), POST (enviar), PUT/DELETE (alterar/apagar).

Migração

Roteiro numerado que cria ou altera tabelas do banco.

OWASP / OWASP Top 10

Fundação de segurança web / sua lista dos 10 maiores riscos.

Parâmetro

Valor enviado junto com a requisição (ex.: `?q=camiseta`).

Porta

Número que identifica um serviço dentro de um servidor.

Prompt

As instruções e dados enviados à IA.

Pydantic

Biblioteca que define moldes de dados e rejeita o que não se encaixa.

Python

A linguagem de programação do projeto.

Redirecionamento

Resposta que manda o navegador para outro endereço.

Redis / Streams

Programa de dados em memória / seu recurso de filas de mensagens.

Rede (Docker)

Rede virtual que define quais containers se enxergam e quais saem para a internet.

Requisição / resposta

O pedido do navegador / o retorno do servidor.

SaaS

Software vendido como serviço pela internet.

Scan passivo / ativo

Só observar / enviar requisições modificadas para testar.

Schema

A estrutura das tabelas do banco (ou de um formato de dados).

SDK

Kit de bibliotecas oficial para usar um serviço (ex.: a API da Claude).

Servidor

Computador que responde aos pedidos dos clientes.

Sessão (ZAP)

O "caderno" de um scan dentro do ZAP; criamos um novo a cada scan.

Severidade

Gravidade de uma falha (crítica a informativa).

SPA

Site montado pelo JavaScript no navegador.

Spider

Rastreador que segue links para descobrir as páginas de um site.

SQL / SQL injection

Linguagem dos bancos de dados / falha que permite alterar as consultas ao banco.

Stack

O conjunto de tecnologias de um site (servidor, linguagem, framework).

Status code

Número que resume o resultado de uma requisição (200, 404, 500...).

Teste unitário / de integração

Testa uma peça isolada / testa as peças funcionando juntas, de verdade.

Token

Código que prova a identidade de alguém; tão sensível quanto uma senha.

URL

Endereço completo de uma página.

uv

Ferramenta que instala o Python e as bibliotecas do projeto.

Validação

Etapa que prova se uma falha é real.

Volume (Docker)

Pasta guardada fora do container, para os dados sobreviverem.

Vulnerabilidade

Defeito que permite a alguém fazer o que não deveria.

Worker

Programa que fica esperando tarefas numa fila e as executa.

WSL

Linux rodando dentro do Windows.

XSS

Falha em que o site devolve código do atacante, que roda no navegador da vítima.

ZAP

Ferramenta da OWASP que testa a segurança de aplicações web; usada na etapa 4.